

Software Performance Analysis Using a Language Measure

Yiguang Hong¹

Kishor S. Trivedi²

Asok Ray

Shashi Phoha

Duke University
Durham, NC 27708, USA

Pennsylvania State University
University Park, PA 16802, USA

Abstract

This paper presents software performance analysis using a finite state automaton model. A signed real measure of formal languages has been used for quantitative evaluation of the software. This paper extends an earlier model based on a discrete time Markov chain to a semi-Markov process. As an example, a software rejuvenation policy is analyzed using the proposed approach.

Keywords: automata, language measure, semi-Markov reward model, software analysis, rejuvenation policy

1 Introduction

Software performance and reliability are demanded increasingly since requirements for constantly functioning software increase dramatically due to the ubiquitous nature of computer systems, communication networks, and large hierarchical systems. Therefore, techniques to improve the software reliability during the design, code development as well as the execution phase become more and more important. One such technique is called software rejuvenation, first proposed in [7], which can be regarded as a preventive maintenance policy to prevent or postpone software failures due to software aging [14]. However, rejuvenation implies an overhead and hence it is important to determine optimal times to rejuvenation and optimal policies. Up to now, most of these techniques fall into the category of open loop rejuvenation. That is, the rejuvenation action is carried out based on random or periodic triggers. Recently, the idea of closed loop rejuvenation has drawn attention, where the rejuvenation is performed based on measured software degradation levels [2] [5].

Automata theory has been widely used in modeling discrete event dynamical systems [6] and has also been extended to closed loop control with combination of the plant and the supervisor as sublanguages to enable restricted legal behavior of the controlled plant [10] [9] [1]. Recently, Wang

and Ray [15] have formulated a signed measure of regular languages and their sublanguages based on a characteristic vector and a cost matrix. This work is followed by Ray and Phoha [11] who have constructed a vector space of formal languages and defined a metric based on the total variation measure of the language. Each marked state is characterized by a signed real value that is chosen based on the designer's perception of the states' impact on the system performance. Each event string is assigned a cost, conceptually similar to the conditional probability, based on the states at which it is generated. This procedure permits an event string terminating at a good (bad) marked state to have a positive (negative) measure; any event string terminating at a non-marked state has zero measure. A discrete-event supervisor can be designed in this setting such that a controlled sublanguage attempts to disable as many bad strings as possible and as few good strings as possible to enhance the performance of the supervised plant. Different supervisors may achieve this goal in different ways and generate a partially ordered set of controlled languages. The language measure creates a total ordering on the performance of the partially ordered set of controlled languages, which provides a precise quantitative comparison of the controlled plant behavior under different supervisors in applications of various areas. This paper extends the scope of the language measure to semi-Markov processes from discrete time Markov chains as in [15] and [11]. Then, we use this language measure to analyze software rejuvenation.

The rest of the paper is organized as follows. In section 2, a brief review of the language measure is provided. Section 3 extends the language measure for an automaton model via semi-Markov reward process with an absorbing state. Following that, a general software rejuvenation policy is analyzed in section 4. The paper is concluded in section 5.

2 Brief review of language measures

This section briefly reviews the concept of signed real measure of regular languages as reported by Wang and Ray [15] and Ray and Phoha [11]. Let $G_i = (Q, \Sigma, \delta, q_i, Q_m)$ be a finite automata model to represent the discrete-event dynamic behavior of a physical plant. Let n denote the cardinality of the state set Q , i.e., $|Q| = n$, and $\mathcal{I} = \{1, \dots, n\}$ the index

¹Dr. Hong is also with Institute of Systems Science, Chinese Academy of Sciences Beijing 100080, China. Email: hong@ee.duke.edu

²This work was done when Dr. Trivedi was a Visiting Professor holding the Poonam and Prabhu Goel Chair in the Department of Computer Science and Engineering at IIT Kanpur

set of Q ; Σ the (finite) alphabet of events; Σ^* is the set of all finite-length strings of events including the empty string ϵ ; $\delta : Q \times \Sigma \rightarrow Q$ is a (possibly partial) function of state transitions and $\delta^* : Q \times \Sigma^* \rightarrow Q$ is an extension of δ ; the state q_i is the initial state; and Q_m is the set of marked states such that, $\emptyset \subseteq Q_m \subseteq Q$. Since δ is allowed to be a partial function, the regular language $L(G_i)$ generated by the DFSA G_i is given as: $L(G_i) \subseteq \Sigma^*$, and $L(G_i) = \Sigma^*$ iff δ is a total function. Therefore, if δ is a partial function, the state set Q can be augmented with an additional non-marked dead-lock (or absorbing) state q_{n+1} , called the dump state [10][9], such that the partial function δ can be extended to a total function $\delta_{ext} : (Q \cup q_{n+1}) \times \Sigma \rightarrow (Q \cup q_{n+1})$. Further details about automata theory can be found in references such as [6] and [10].

Definition 1. The language $L(G_i)$ generated by an automaton G_i initialized at the state $q_i \in Q$ is defined as:

$$L(G_i) = \{s \in \Sigma^* \mid \delta^*(q_i, s) \in Q\} \quad (1)$$

The language $L_m(G_i)$ marked by G_i initialized at the state $q_i \in Q$ is defined as:

$$L_m(G_i) = \{s \in \Sigma^* \mid \delta^*(q_i, s) \in Q_m\} \quad (2)$$

The set Q_m of marked states is partitioned into Q_m^+ and Q_m^- , i.e., $Q_m = Q_m^+ \cup Q_m^-$ and $Q_m^+ \cap Q_m^- = \emptyset$, where Q_m^+ contains all good marked states that we desire to reach, and Q_m^- contains all bad marked states that we want to avoid, though it may not always be possible to completely avoid the bad states while attempting to reach the good states. In general, the marked language $L_m(G_i)$ consists of both good and bad event strings that, starting from the initial state q_i , respectively lead to Q_m^+ and Q_m^- . Any event string belonging to the language $L^0 = L(G_i) - L_m(G_i)$ leads to one of the non-marked states belonging to $Q - Q_m$ and L^0 does not contain any one of the good or bad strings. To achieve the above goal of signed measure, we characterize the marked states such that each state in Q_m^+ is assigned a positive weight and each state in Q_m^- a negative weight; and each unmarked state is assigned the zero weight. The weights are chosen by the designer based on the perception of each marked state's role in the system performance.

Definition 2. For every $q_k \in Q$, let $L(q_k, q_i)$ denote the set of all strings that, starting from the state q_i , terminate at the state q_k , i.e.,

$$L(q_k, q_i) = \{s \in \Sigma^* \mid \delta^*(q_i, s) = q_k \in Q\} \quad (3)$$

Based on the equivalence classes defined in the Myhill-Nerode Theorem [6], the regular languages $L(G_i)$ and $L_m(G_i)$ can be expressed as:

$$L(G_i) = \bigcup_{q_k \in Q} L(q_k, q_i) = \bigcup_{k=1}^n L(q_k, q_i) \quad (4)$$

$$L_m(G_i) = \bigcup_{q_k \in Q_m} L(q_k, q_i) = L_m^+ \cup L_m^- \quad (5)$$

where the sublanguage $L(q_k, q_i) \subseteq G_i$ having the initial state q_i is uniquely labeled by the terminal state $q_k, k \in \mathcal{I}$ and $L(q_j, q_i) \cap L(q_k, q_i) = \emptyset \forall j \neq k$; and $L_m^+ \equiv \bigcup_{q_k \in Q_m^+} L(q_k, q_i)$ and $L_m^- \equiv \bigcup_{q_k \in Q_m^-} L(q_k, q_i)$ are good and bad sublanguages of $L_m(G_i)$, respectively. Then, $L^0 = \bigcup_{q_k \notin Q_m} L(q_k, q_i)$ and $L(G_i) = L^0 \cup L_m^+ \cup L_m^-$.

Definition 3. Let Θ be a σ -algebra of $L(G_i)$. Then, the set function $\mu : \Theta \rightarrow \mathbf{R} \equiv (-\infty, +\infty)$, is called a signed real measure if the following two conditions are satisfied [12]:

- (1) $\mu(\emptyset) = 0$;
- (2) $\mu(\bigcup_{j=1}^{\infty} K_j) = \sum_{j=1}^{\infty} \mu(K_j) \forall K_j \in \Theta$ and $K_j \cap K_k = \emptyset$ if $j \neq k$.

Now we define a signed real measure $\mu : 2^{L(G_i)} \rightarrow \mathbf{R}$ for a given automata such that:

$$\forall q \in Q, \quad \mu(L(q, q_i)) \begin{cases} = 0, & q \notin Q_m \\ > 0, & q \in Q_m^+ \\ < 0, & q \in Q_m^- \end{cases} \quad (6)$$

The next definition is an extension of Definition 2.4 in [15] by introducing reward rate functions and sojourn time functions for automata. These functions are defined for a semi-Markov reward process, which is used in the following discussions.

Definition 4. The characteristic function $\chi^0 : Q \rightarrow [-1, 1]$ that assigns a signed real weight to state-based sublanguages $L(q, q_i)$ is defined as:

$$\forall q \in Q, \quad \chi^0(q) \in \begin{cases} [-1, 0), & q \in Q_m^- \\ \{0\}, & q \notin Q_m \\ (0, 1], & q \in Q_m^+ \end{cases} \quad (7)$$

Also we can define the rate function $\chi^r : Q \rightarrow [-1, 1]$ in the same way. Moreover, we define the expected sojourn time function $\tau : Q \rightarrow [0, +\infty)$ as the mean amount of time for the system to stay in a given state. Then the state weighting vector, denoted by $\mathbf{X} = [\chi_1 \chi_2 \cdots \chi_n]^T$, where $\chi_k \equiv \chi^0(q_k) + \chi^r(q_k)\tau(q_k) \forall k$, is called the $\mathbf{X}$ -vector. The k -th element χ_k of $\mathbf{X}$ -vector is the weight assigned to the corresponding terminal state q_k .

Moreover, according to [15], we can define the signed real measure of the language of an automaton G_i initialized at a state $q_i \in Q$ as:

$$\mu_i \equiv \mu(L(G_i)) = \sum_{q \in Q} \mu(L(q, q_i)) = \mu(L_m(G_i)) \quad (8)$$

with $\mu(L(q, q_i)) = 0$ for any $q \notin Q_m$. In other words, μ_i is the signed measure of the marked language $L_m(G_i)$ of G_i .

Definition 5. Let $L_i \equiv L_m(G_i), i \in \mathcal{I}$, denote the regular expression representing the marked language of an n -state automaton $G_i = (Q, \Sigma, \delta, q_i, Q_m)$ where q_i is the initial state. Moreover, let σ_j^k denote the set of event(s) $\sigma \in \Sigma$ that is defined on the state q_j and leads to the state $q_k \in Q$, where $j, k \in \mathcal{I}$, i.e., $\delta(q_j, \sigma) = q_k, \forall \sigma \in \sigma_j^k \subseteq \Sigma$.

To compute the measure of the language $L(q, q_i)$, we assign a cost to each string terminating at the state q starting from the initial state q_i . To this end, the event cost is defined, conceptually similar to the transition probability as in [15].

3 Measure computation for semi-Markov processes

This section extends the scope of the language measure to semi-Markov reward processes (SMP) from discrete-time Markov chains [15] and [11] to generalize the state transition distributions. Let an automaton $G_i = (Q, \Sigma, \delta, q_i, Q_m)$ have a deadlock (or absorbing) state $q_{n+1} \in Q$ where there are only transitions from other states to q_{n+1} but there are no transitions from q_{n+1} to other states. Like what is done for a dump state [9], the absorbing state $q_{n+1} \notin Q_m$. As mentioned before, the underlying process of the automaton under consideration is a semi-Markov process, with general, rather than geometric, state transition distributions, which is an mathematical extension of the conventional discrete time Markov chain.

In order to convert the symbolic equations of automata into a set of algebraic equations, we introduce the one-step $(n+1) \times (n+1)$ transition probability matrix K of the embedded Markov chain of SMP with an absorbing state as [8] [3] [13]:

$$K = \begin{pmatrix} \mathbf{\Pi} & \Delta \\ 0 & 1 \end{pmatrix}$$

where the $n \times n$ state transition cost matrix $\mathbf{\Pi}$ is defined as:

$$\mathbf{\Pi} = \begin{bmatrix} \pi_{11} & \pi_{12} & \dots & \pi_{1n} \\ \pi_{21} & \pi_{22} & \dots & \pi_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \pi_{n1} & \pi_{n2} & \dots & \pi_{nn} \end{bmatrix}$$

and the matrix $\Delta \in R^n$ contains non-zero elements, representing the probabilities of some of the unmodeled events generated at the corresponding states. This suffices for the matrix $(\mathbf{I} - \mathbf{\Pi})^{-1}$ to be a bounded linear operator. In other words, the absorbing state guarantees the boundedness of $(\mathbf{I} - \mathbf{\Pi})^{-1}$.

The state transition cost of G_i is defined as a function $\pi : Q \times Q \rightarrow [0, 1]$, where $\pi_{ij} = \pi[q_j|q_i]$ denotes the probability of all the one-step transition cases from q_i to q_j . $\pi_{ij} = 0$ if $\{\sigma \in \Sigma : \delta(q_i, \sigma) = q_j\} = \emptyset$. The expected sojourn time

(in Definition 4) in state q_i ($i = 1, \dots, n$) of the SMP is denoted by τ_i . Both π_{ij} and τ_i ($i = 1, \dots, n; j = 1, \dots, n$) can be calculated [8].

It is necessary to obtain a regular expression which explicitly yields the state-based event sequences. As in [11] [15], a procedure for language measure computation is presented below. Given an automaton $G_i = (Q, \Sigma, \delta, q_i, Q_m)$ with an absorbing state $q_{n+1} \notin Q_m$, where $i = 1, \dots, n$, we proceed to obtain the system of equations by a set of regular expressions L_i of the marked language $L_m(G_i), i = 1, \dots, n$ (We do not take the absorbing state as the initial state, that is, $i \neq n+1$. Otherwise, the corresponding languages will be trivial), as follows:

$$\forall q_i \in Q, \quad L_i = \sum_j R_{i,j} + \mathcal{E}_i, \quad i = 1, \dots, n \quad (9)$$

where $\forall i \in \mathcal{I}$, $R_{i,j}$ is defined as:

1. If $\exists \sigma \in \Sigma$, such that $\delta(q_i, \sigma) = q_j \in Q, j \in \mathcal{I}$, then $R_{i,j} = \sigma_i^j L_j$, otherwise, $R_{i,j} = \emptyset$.
2. If $q_i \in Q_m, \mathcal{E}_i = \epsilon$, otherwise, $\mathcal{E}_i = \emptyset$.

According to Definition 4, we have

$$\forall i \in \mathcal{I}, \quad \mu(\mathcal{E}_i) = \begin{cases} \chi_i & \text{if } \mathcal{E}_i = \epsilon \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

Moreover, since the automaton model is modeled to be semi-Markov, $\mu(\sigma_i^j L_j) = \mu(\sigma_i^j) \mu(L_j)$. Therefore, $\mu(\sigma_i^j L_j) = \pi[q_j|q_i] \mu(L_j) = \pi_{ij} \mu(L_j)$.

As shown in [15], the set of symbolic equations may be written as:

$$L_i = \sum_j \sigma_i^j L_j + \mathcal{E}_i, \quad i = 1, \dots, n \quad (11)$$

We note the following special cases.

- (1) If $\mathcal{E}_i = \emptyset, \forall L_i$, then $L_m(G) = \emptyset$. This implies that the automaton has no marked state.
- (2) If $\exists q_i \in Q$ such that $L_i = \epsilon$, then q_i is marked. Furthermore, q_i is a deadlock state.

Now we present the language measure in terms of the state transition cost matrix $\mathbf{\Pi}$ as delineated below. This is a theoretical result, extending Theorem 3.1 of [15] to a semi-Markov case.

Theorem 1. The language measure of the symbolic equations (11) is given by

$$\mu_i = \sum_j \pi_{ij} \mu_j + \chi_i, \quad i = 1, \dots, n \quad (12)$$

or in vector notation: $\mu = \Pi\mu + X$ whose solution is given by:

$$\mu = (I - \Pi)^{-1}X \in \mathbb{R}^n \quad (13)$$

(The language measure vector, $\mu = [\mu_1 \ \mu_2 \ \cdots \ \mu_n]$, is called the μ -vector). Moreover, $\mu_{n+1} = 0$.

Proof: The proofs of (12) and (13) are given in [15]. In addition, $\chi(q_{n+1}) = 0$ because $q_{n+1} \notin Q_m$. Therefore, $\mu_{n+1} = 0$, which is also consistent with (12).

In fact, the theorem facilitates the measure computation without going through the symbolic equations. The example in next section illustrates this procedure.

4 Software rejuvenation

This section presents the software rejuvenation problem to illustrate the supervisory control concept and its validation using the measure-theoretic procedure.

Software rejuvenation, carried out to improve software performance and reliability, has recently drawn attention. It is well known that, without any rejuvenation, the software is likely to experience aging. That is, the software operation may gradually degrade with time due to the depletion of system resources. As an example, we use the amount of free physical memory as the resource that degrades. In this paper, a rejuvenation scheme is given with its actions offered at two levels: *Level-1* is a service-level rejuvenation, where services are written in a way that a stoppage of that service saves any necessary data and the corresponding restart of that service brings the application to a more usable state (so called ‘partial restart’). *Level-2* is a system-level rejuvenation, that is, a reboot to restart the overall system in order to reclaim all the total amount of free physical memory (so called ‘full restart’). Note that the system will crash when the resource is exhausted. In other words, with no rejuvenation action applied to the software system, the system resources may naturally be exhausted and consequently the performance such as the quality of service will be degraded. With level-1 restart, the free memory situation will be improved in a minimal way, while with level-2 restart, the resources will be released almost completely.

Up to now, various rejuvenation strategies, open loop or closed loop, have been proposed and analyzed by means of well known paradigms such as Markov chains, non-Markovian models, measurement-based approaches ([7] [2] [4] [5] and the references therein). In this paper, we investigate software rejuvenation using a language measure of (timed) automata, which is viewed as one of the modeling and analysis methods in computer science. In the sequel, we show that the approach proposed in the preceding sections is effective and implementable in the study of software

availability.

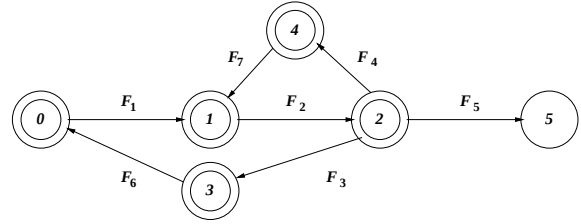


Figure 1: Software rejuvenation

The automaton model is depicted as shown in Figure 1. It is well known that memory leaks, unreleased object references, erroneous pointer handling, and roundoff errors are typical sources of software faults that result in aging of the software executive. In other words, the working state of the software system is also a degradation process. The description of each state of the DFSA model is listed as follows:

- **State 0:** This is the fully operational software execution phase. Only level-2 restart can make the system go back to this state. The system works well in this state. So, the state is *available* to operate the workload.
- **State 1:** This is the degraded software execution phase. Partial restart can make the system go back to this state. The system still works in this state. So the state is also *available* to operate the workload.
- **State 2:** In this state, the software is in its highly degraded execution phase or alert phase. It is still *available* to operate the workload, but a rejuvenation action or a system crash may happen in this state.
- **State 3:** In this state, the level-2 restart is being carried out to rejuvenate the software system, which renders it *unavailable* to operate the workload.
- **State 4:** In this state, the level-1 restart is being carried out and the software is *unavailable* to operate the workload.
- **State 5:** In this state, the system crashes (then needs to be rebooted later). It is the only absorbing state.

Therefore, we have a DFSA model with the set Q consisting of states 0 up to 5 and the set Q_m of marked states 0 to 4. The initial state is state 0 and the alphabet Σ consists of the events with distributions $F_j, j = 1, \dots, 7$. Among the marked states (in Q_m), the available states (i.e., states 0, 1, and 2) are good with positive assigned weights (or equivalently, Q_m^+ consists of states 0, 1, and 2), while the unavailable states (i.e., states 3 and 4) are bad with negative

weights, forming Q_m^- . The absorbing state 5 is non-marked and is assigned zero weight.

In the rejuvenation design, efforts are made for improvement of software availability, i.e., to keep the software in available states as long as possible (before the system crashes). This is related to the sojourn times, and the rejuvenation costs being as small as possible; this being reflected in the reward rates. For different parameters assigned to the software rejuvenation scheme, different rejuvenation policies are optimal. With the language measure, it is now convenient to evaluate these policies as seen later in this section.

When the software in operation stays in state 0 (or state 1), it will enter state 1 (or state 2) with a general distribution function $F_1(t)$ (or $F_2(t)$). When it stays in state 2, it may enter state 3 with a general distribution function $F_3(t)$ (full restart), or enter state 4 with a general distribution function $F_4(t)$ (partial restart), or fail with a distribution function $F_5(t)$ (in state 5). The distribution function for the duration of restart is $F_6(t)$ (in state 3), while the function for that of partial restart is $F_7(t)$ (in state 4). Since each state is a regenerative state, the underlying stochastic process is semi-Markov. According to the standard procedure of SMP analysis [8], the transition probability matrix K of the embedded Markov chain of the SMP is:

$$K = [\pi_{ij}]_{6 \times 6} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & r & p & k \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{\Pi} & \Delta \\ 0 & 1 \end{pmatrix}$$

where π_{ij} ($i \neq j$, $i, j = 0, \dots, 5$) is the probability that if the SMP has just entered state i , the next transition will be to state j :

$$k = \pi_{25} = \int_0^\infty \bar{F}_3 \bar{F}_4 dF_5(t),$$

$$p = \pi_{24} = \int_0^\infty \bar{F}_3 \bar{F}_5 dF_4(t),$$

and

$$r = \pi_{23} = \int_0^\infty \bar{F}_4 \bar{F}_5 dF_3(t)$$

with $p + r = 1 - k$ [8]. Also, the SMP expected sojourn times in states 0, 1, 2, 3, and 4, denoted by τ_j , $j = 0, \dots, 4$, respectively, can be obtained based on F_j , $j = 1, \dots, 7$ [8]. The expected sojourn times in state 0 and state 1 are

$$\tau_0 = \int_0^\infty \bar{F}_1(t) dt, \quad \tau_1 = \int_0^\infty \bar{F}_2(t) dt,$$

respectively, while that in state 2 is

$$\tau_2 = \int_0^\infty \bar{F}_3(t) \bar{F}_4(t) \bar{F}_5(t) dt$$

The expected sojourn time in states 3 and 4 are

$$\tau_3 = \int_0^\infty \bar{F}_6(t) dt, \quad \tau_4 = \int_0^\infty \bar{F}_7(t) dt$$

The fundamental matrix based on $\mathbf{\Pi}$ -matrix can be calculated as [13]:

$$M = [m_{ij}]_{5 \times 5} = (\mathbf{I} - \mathbf{\Pi})^{-1} = \begin{pmatrix} 1 + m_2 & 1 + m_1 + m_2 & 1 + m_1 + m_2 & m_1 & m_2 \\ m_2 & 1 + m_1 + m_2 & 1 + m_1 + m_2 & m_1 & m_2 \\ m_2 & m_1 + m_2 & 1 + m_1 + m_2 & m_1 & m_2 \\ m_2 & 1 + m_1 + m_2 & 1 + m_1 + m_2 & 1 + m_1 & m_2 \\ m_0 & 1 + m_1 + m_2 & 1 + m_1 + m_2 & m_1 & m_0 \end{pmatrix}$$

where

$$m_0 = \frac{1-r}{k}, \quad m_1 = \frac{r}{k}, \quad m_2 = \frac{p}{k}$$

Note that m_{ij} , for $0 \leq i, j \leq 4$, is actually the expected value of the number that the procedure visits state j before entering the failure state, given that it starts in state i [13].

To evaluate different rejuvenation policies corresponding to different r, p , and k , we assume that the reward rate vector is $[\chi_0, \chi_1, \chi_2, \chi_3, \chi_4] = [0.5, 0.5, 0.2, -1, -0.5]$ considering the marked states 0, 1, and 2 are good. Because the cost of the level-1 rejuvenation is less than the cost of the level-2 rejuvenation, the reward rate of the level-1 rejuvenation (denoted by χ_4) is greater than that of the other (denoted by χ_3). By Definition 4 (noting that the characteristic function $\chi^0 = 0$ in our case), the reward weighting vector is

$$\mathbf{X} = [\chi_0 \tau_0, \chi_1 \tau_1, \chi_2 \tau_2, \chi_3 \tau_3, \chi_4 \tau_4]^T.$$

Therefore, the language measure can be used in the evaluation of software rejuvenation policies. For a simple illustration, we fix k and take r as the only control parameter (for $p = 1 - r - k$). Note that $p = 0$ means that rejuvenation actions are only level-2 restarts, while $r = 0$ leads to that the rejuvenation policy only consists of partial restart actions. Our aim is to enhance the weighted execution time, which can be evaluated by the scalar performance $\mu_0 = \mu(L(G_0))$ because state 0 is the initial state, as discussed in Section 3. In this way, the optimal software rejuvenation policy of the given rejuvenation scheme, viewed as an optimal supervisor, can be obtained by adjusting the control parameter r .

Following Theorem 1, we have

$$\mu_0 = \sum_{0 \leq j \leq 4} m_{0j} \chi_j \tau_j$$

Therefore,

$$\mu_0 = \chi_0 \tau_0 + \frac{1}{k} (\chi_1 \tau_1 + \chi_2 \tau_2)$$

$$+\frac{1-k}{k}\chi_4\tau_4+r\frac{\chi_0\tau_0+\chi_3\tau_3-\chi_4\tau_4}{k} \quad (14)$$

With (14), we are in a position to calculate the optimal value of r . Roughly speaking, if $\chi_0\tau_0 + \chi_3\tau_3 - \chi_4\tau_4 = 0.5\tau_0 - \tau_3 + 0.5\tau_4 > 0$, then taking $r = 1 - k$ (i.e., $p = 0$, pure level-2 restart actions) will maximize μ_0 ; if $\chi_0\tau_0 + \chi_3\tau_3 - \chi_4\tau_4 < 0$, then taking $r = 0$ (i.e., only partial restart actions are carried out) will maximize μ . Employing the experimental data from [5], we have $\tau_0 \approx 250\text{min}$, $\tau_3 \approx 3\text{min}$, and $\tau_4 \approx 5\text{min}$. Therefore, the optimal rejuvenation policy consists of level-2 restarts (or full restarts) without any partial restarts (because $\chi_0\tau_0 + \chi_3\tau_3 - \chi_4\tau_4 > 0$ and then $p = 0$). This conclusion is consistent with that obtained using a different approach in [5]. It is also worthwhile to point out that the (timed) automata model given here is more general than that in [5], which were mainly based on deterministic analysis.

5 Conclusions

This paper addresses the issue of software availability analysis using a signed real measure of regular languages and their sublanguages. This approach requires the software operations to be modeled as a (timed) finite-state automaton for representation as a regular language. Specifically, the language measure analysis of automata is given by virtue of semi-Markov reward process instead of discrete time Markov chain [15] to generalize the sojourn time distributions. Positive weights are assigned to good marked states and negative weights to bad marked states. Then, a software rejuvenation process is analyzed to illustrate the usage of the proposed language-measure approach. Even with a model generalized from that in [5], the analysis is still effective and simple using this approach.

6 Acknowledgments

This research was supported in part by the Army Research Office under MURI Grant No. C-DAAD19 01-1-0646, in part by the Air Force Office of Scientific Research under MURI Grant No. F49620-00-1-0327, and in part by NSF of China. Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the author(s) and does not necessarily reflect the view of the sponsoring agencies.

References

[1] C. Cassandras, *Discrete Event Systems: Modeling and Performance Analysis*, Aksen Associates, Boston, MA, 1993.

[2] V. Castelli, R. Harper, P. Heidelberger, S. Hunter, K. Trivedi, K. Vaidyanathan and W. Zeggert, Proactive management of software aging, *IBM Journal of Research and Development*, Vol. 45, No. 2, March 2001.

[3] G. Ciardo, R. Marie, B. Sericola, and K. Trivedi, Performability analysis using semi-Markov reward processes, *IEEE Trans. on Computers*, 39, 10, 1251-1264, 1990.

[4] S. Garg, A. Puliafito, M. Telek, and K. Trivedi, Analysis of preventive maintenance in transactions based software systems, *IEEE Trans. on Computers*, vol. 47, 96-107, 1998.

[5] Y. Hong, D. Chen, L. Li, and K. Trivedi, Closed loop design for software rejuvenation, *Workshop on self healing, adaptive, and self-managed systems*, New York, NY, June 2002.

[6] J. Hopcroft, R. Motwani, and J. Ullman, *Introduction to Automata Theory, Language, and Computation*, 2nd ed., Addison Wesley, 2001.

[7] Y. Huang, C. Kintala, N. Kolettis, and N. Fulton, Software rejuvenation: analysis, module, and applications, *Proc. of 25th Int. Symposium on Fault-tolerance Computing*, 1995.

[8] V. Kulkarni, *Modeling and Analysis of Stochastic Systems*, Chapman & Hall, London, 1995.

[9] R. Kumar and V. Garg, *Modeling and Control Logical Discrete Event Systems*, Kluwer Academic, 1995.

[10] P. Ramadge and W. Wonham, Supervisory control of a class of discrete event processes, *SIAM Journal of Control and Optimization*, vol. 25, no. 1, 206-230, 1987.

[11] A. Ray and S. Phoha, A language measure for discrete event automata, *International Federation of Automatic Control World Congress*, Barcelona, Spain, July 2002.

[12] W. Rudin, *Real and Complex Analysis*, 3rd ed., McGraw-Hill, 1987.

[13] K. Trivedi, *Probability & Statistics with Reliability, queuing, and Computer Science Applications*, 2nd ed., John Wiley & Sons, 2001.

[14] K. Vaidyanathan and K. Trivedi, Extended classification of software faults based on aging, *Proc. IEEE Int'l. Symp. on Software Reliability Engineering*, Hong Kong, November 2001.

[15] X. Wang and A. Ray, Signed real measure of regular languages, *American Control Conference*, Anchorage, Alaska, May 2002.